Introduction to Software Engineering

# Software Requirements Specification




Software Engineering Department
Faculty of Information and Technology
University of Science

# Table of Contents

**1 Member Contribution Assessment** **1**
**2 Introduction** **2**
2.1 Purpose and Scope 2
2.2 Glossary 3
**3 Problem Statement** **4**
3.1. Business Context 4
3.2. Current Problems 4
3.3. Goals 5
3.4. Operating Environment 5
3.5. Design and Implementation Constraints 6
**4 Stakeholders** **7**
**5 Requirements** **9**
5.1 Functional Requirements 9
5.2 Non-Functional Requirements 11
5.3 Domain Requirements and Business Rules 12
5.4 Acceptance Criteria 13
**6 Use Case Model** **15**
6.1 Use Case Diagram 15
6.2 Actors 16
6.3 Use Case List 16
**7 Use Case Specifications** **18**
7.1 Use Case UC-01 18
7.2 Use Case UC-02 19
7.3 Use Case UC-03 19
7.4 Use Case UC-04 20
7.5 Use Case UC-05 21
7.6 Use Case UC-06 22
7.7 Use Case UC-07 23
7.8 Use Case UC-08 24
7.9 Use Case UC-09 25
**8 Prototype / Mockups** **27**
**9 Requirements Traceability Matrix** **29**
**10 Requirements Review** **31**
**11 AI Usage Log** **32**

# 1 Member Contribution Assessment

| ID | Name | Role(s) | Contribution (%) | Evidence |
|---|---|---|---|---|
| 24127552 | Nguyễn Võ Nhật Thuận | Team lead / Business analyst / Full-stack developer | 20% | Define domain rules & acceptance criteria<br><br>Write Detailed Use Case Specifications |
| 24127232 | Lê Phụng Sơn | Business analyst / Team lead backup / Full-stack developer | 20% | Write Functional Requirements (FR) & NFRs<br><br>Write Software Requirements Specification Introduction (Purpose and Scope + Glossary) |
| 24127087 | Dương Nguyên Nam | Designer / Full-stack developer / Business Analyst | 20% | Draw Use Case Diagram<br><br>Write Software Requirements Specification Stakeholders<br><br>Write Software Requirements Specification Use Case Model (Actors + Use Case List) |
| 24127106 | Lai Phạm Như Phúc | Full-stack developer / Tester | 20% | Design Interactive Wireframes/Figma Mockups |
| 24127086 | Trần Hoàng Khánh My | Tester / Designer / Full-stack developer | 20% | Build Requirements Traceability Matrix<br><br>Write Software Requirements Specification Problem Statement |

# 2 Introduction

## 2.1 Purpose and Scope

### Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements of SmartLib Booking, a web-based study-room reservation system for a university library. It serves as a reference for the development team and stakeholders during system design, implementation, testing, and evaluation.

### Scope

SmartLib Booking streamlines library room reservations by replacing manual Google Sheets scheduling with an integrated web application.

The system allows students to search for rooms, submit and manage reservations, and receive email notifications. Librarians can manage room inventories, handle booking requests manually, or enable automatic first-come-first-served (FCFS) approval. An optional locally hosted AI assistant supports natural-language room searches and floor-plan inquiries.

The system is designed for a single university library and is accessible through modern web browsers on desktop and mobile devices.

The following are outside the project scope:

- Online payments and fee or fine settlement.
- Native mobile applications.
- Multi-library or multi-campus support.
- Cloud-hosted AI services.

### Intended Readers

This document is intended for the project sponsor (course instructor and teaching assistant), development team, students, librarians, and library management. It helps the development team implement and test the system while enabling stakeholders to understand its capabilities and limitations.

## 2.2 Glossary

| Term | Definition |
|---|---|
| SmartLib Booking | A web-based study-room reservation system for a single university library. |
| SRS | Software Requirements Specification; a document defining software requirements and constraints. |
| Student | A user who searches for rooms and manages reservations. |
| Librarian | A staff member who manages room inventories and reservation requests. |
| Room Inventory | The collection of room profiles and their attributes, such as capacity, location, and equipment. |
| Reservation | A request to use a study room during a specified time slot. |
| Booking Conflict | An overlap between a requested time slot and an existing reservation for the same room. |
| Manual Review Mode | A workflow in which librarians review and approve reservation requests. |
| FCFS (First-Come-First-Served) | An approval mode that automatically approves valid, conflict-free requests in arrival order. |
| Email Notification | An email informing users of reservation events, such as submission, approval, or cancellation. |
| AI Assistant | An optional tool for natural-language room searches and floor-plan inquiries. |
| VLM (Vision-Language Model) | An AI model that processes visual information and natural-language input. |
| Authentication | The process of verifying a user's identity before granting system access. |
| Single-Tenant System | A system designed to serve one library organization. |

# 3 Problem Statement

## 3.1. Business Context

The increasing demand for study rooms, group discussion areas, and quiet study spaces in university libraries creates a need for an efficient reservation management system. Currently, the room booking process relies heavily on manual operations, making it difficult for students and library staff to manage reservations effectively.

SmartLib Booking is proposed as a web-based room reservation system for university libraries. It aims to simplify the booking process, reduce administrative workload, and improve the overall efficiency of room management.

## 3.2. Current Problems

The existing reservation process requires students to submit booking requests and librarians manually review requests and update room schedules using a shared Google Sheet. This approach presents several challenges:

- Time-consuming procedures: Manual verification and approval can delay booking confirmations, particularly during periods of high demand.
- Scheduling conflicts: Manually maintained schedules are prone to human errors, potentially resulting in overlapping reservations.
- Limited availability visibility: Students cannot easily check room availability, capacity, equipment, or booking status through a centralized interface.
- Administrative workload: Librarians must manually manage reservations, update room information, and handle schedule changes or cancellations.

These limitations reduce the efficiency and reliability of the current reservation process.

## 3.3. Goals

SmartLib Booking aims to replace the existing manual workflow with a centralized and automated reservation system.

The system will allow students to search for available rooms based on date, time, capacity, location, and equipment; submit reservation requests; and view, cancel, or reschedule their bookings.

Librarians will be able to manage room profiles, configure maintenance periods, and review booking requests. The system will support both manual approval and an optional first-come-first-served automatic approval mode, with conflict detection to prevent overlapping reservations.

Additional features include automated email notifications for booking status updates and an optional AI assistant powered by a locally hosted Vision-Language Model (VLM) to support natural-language room searches, floor-plan inquiries, and booking assistance.

## 3.4. Operating Environment

SmartLib Booking will operate as a web application accessible through modern browsers, including Chrome, Safari, Firefox, and Edge, on desktop and mobile devices with an internet connection.

The backend will run on a Python-based web server using Django, while PostgreSQL will manage reservation and room data. The production database will be hosted on Supabase Cloud, and local development will use Docker containers.

The optional AI assistant will operate through a locally hosted VLM service on a librarian's workstation, connected to the main application through an API.

## 3.5. Design and Implementation Constraints

The system must comply with the following constraints:

- Technology constraints: The application will use Python and Django for the backend, Django Templates and Tailwind CSS for the frontend, and PostgreSQL for data storage.
- Authentication and authorization: Users must authenticate with valid student or staff credentials, and system operations must respect their assigned roles.
- Booking policies: Reservations must follow library rules, including a maximum of 8 booking hours per week, 4 hours per day, and 4 hours per session. Bookings must occur within official operating hours, avoid overlapping reservations, and respect room maintenance periods.
- Deployment constraints: The system is designed for a single university library and will be delivered exclusively as a web application.
- AI resource constraints: The optional VLM must run locally within the available GPU and RAM capacity of staff workstations, without relying on cloud-hosted AI services.
- Scope limitations: Native mobile applications, multi-library or multi-campus management, online payments, and fine settlement systems are excluded from the project scope.

# 4 Stakeholders

| # | Stakeholder | Description | Interests / concerns |
|---|---|---|---|
| 1 | Project sponsor (course instructor & TA) | Evaluate the project, the team's teamwork and the development process. | A complete, well-documented product that meets the PA requirements. Traceability from requirements to use cases, design and tests (Section 9). Correct use of tools and full disclosure of AI use under the course GenAI policy. |
| 2 | Student (end user) | University students who book rooms for individual or group study. Primary actor of UC-02, UC-03, UC-05, UC-06 and UC-10 (use case with "must" priority) | Finding available rooms and their equipment quickly (UC-02). Booking with fast feedback, and instant approval when auto-approval is on (UC-03, UC-12). Viewing, cancelling and rescheduling their own bookings (UC-05, UC-06, UC-10). Email notices on submission, approval and cancellation (UC-13). Clear, fair booking limits of 4 hours per day and 8 hours per week (BR-01, BR-02). No double bookings once a booking is confirmed (BR-05). Being told promptly if the library cancels a booking for maintenance (BR-06). Optional natural-language room search (UC-14 to UC-16). |
| 3 | Librarian / library staff (end user) | Staff who manage rooms and process booking requests. Primary actor of UC-07, UC-08, UC-09 and UC-11 (use case with "must" priority) | Managing room profiles and maintenance status without manual tracking (UC-08, UC-09). Reviewing requests in one queue (UC-07). Turning auto-approval on or off at peak times (UC-11). No double bookings (BR-05) and less work than the Google Docs and Google Sheets process. A dashboard that feels like the familiar spreadsheet layout (risk R5 in PA1). An AI assistant that runs on a staff workstation without slowing it down or crashing it. |

| | | | |
|---|---|---|---|
| 4 | Library management | Decision makers responsible for library services and policies. They are not a system actor and do not use the system directly. | Setting the booking policies the system must enforce: weekly and daily limits, required booking information, and the maintenance override (BR-01, BR-02, BR-03, BR-06). Operating hours as defined in the library schedule (BR-04). Better room use, fewer scheduling conflicts, lower staff workload and low operating cost (free-tier hosting, no cloud AI fees). |
| 5 | Development team (TPNSM) | Five students who analyse, design, build, test and deploy the system. | A feasible scope and a manageable workload (about 3.5 to 4.5 hours per member per week). A working demo for the final presentation. A target grade of 9.0 or higher. Keeping to the 11-week schedule by building the Must-have features first (risks R3 and R4 in PA1). |

# 5 Requirements

## 5.1 Functional Requirements

| ID | Requirement | Feature | Priority | Source |
|---|---|---|---|---|
| FR-01 | The system shall authenticate students and librarians using university credentials through university Identity Provider and enforce role-based access. | Login | Must | Document |
| FR-02 | The system shall allow users to search for available rooms by date, time slot, capacity, and attributes. | Room Search | Must | Document |
| FR-03 | The system shall allow students to submit room bookings with the required information and receive booking confirmation by email. | Room Booking | Must | Document |
| FR-04 | The system shall check for overlapping bookings and room maintenance periods before confirming a booking. | Conflict Checking | Must | Document |
| FR-05 | The system shall display students' past, current, and cancelled bookings with their details. | Booking Management | Must | Document |
| FR-06 | The system shall allow students to cancel their own upcoming pending or approved bookings and release the reserved time slot. | Booking Management | Must | Document |

| | | | | |
|---|---|---|---|---|
| FR-07 | The system shall allow librarians to review pending booking requests and approve or reject them. | Booking Approval | Must | Stakeholder |
| FR-08 | The system shall allow librarians to add, edit, and disable room profiles. | Room Inventory | Must | Stakeholder |
| FR-09 | The system shall allow librarians to set room maintenance periods and cancel affected bookings. | Room Maintenance | Must | Document |
| FR-10 | The system shall allow students to reschedule bookings. | Booking Management | Should | Document |
| FR-11 | The system shall allow librarians to enable or disable automatic booking approval. | Auto-Approval | Should | Document |
| FR-12 | The system shall automatically approve booking requests when automatic approval is enabled. | Auto-Approval | Should | Document |
| FR-13 | The system shall send email notifications for booking-related events. | Notifications | Should | Document |
| FR-14 | The system shall allow users to search for rooms using natural-language queries. | AI Assistant | Should | Document |
| FR-15 | The system shall allow users to ask questions about library floor plans. | AI Assistant | Should | Document |

| | | | | |
|---|---|---|---|---|
| FR-16 | The system shall allow students to book rooms through the AI assistant. | AI Assistant | Should | Document |

## 5.2 Non-Functional Requirements

| ID | Category | Requirement (measurable) | How it will be verified |
|---|---|---|---|
| NFR-01 | Performance | Authentication latency shall be under 500 ms. | Measure authentication latency. |
| NFR-02 | Performance | Room-search queries shall return results within 1 second under normal server load. | Measure search response time under normal server load. |
| NFR-03 | Performance | Booking-conflict checks shall execute within 100 ms. | Measure conflict-check execution time. |
| NFR-04 | Performance | The My Bookings page shall load within 1 second. | Measure page-load time. |
| NFR-05 | Performance | Released booking slots shall be reflected in the search index synchronously within 1 second after cancellation. | Measure the time until the cancelled slot becomes available in search. |
| NFR-06 | Reliability | Booking transactions shall enforce database atomicity. | Test booking transactions and verify that failures do not leave partial transaction changes. |
| NFR-07 | Consistency | Administrative booking-queue updates shall be reflected in real time. | Observe the booking queue after a request is processed. |
| NFR-08 | Consistency | Room-profile updates shall take immediate effect across search endpoints. | Update a room profile and verify the resulting search data. |
| NFR-09 | Performance | Maintenance-related cancellation emails shall be queued within 1 hour. | Compare booking-cancellation timestamps with email-queue timestamps. |

## 5.3 Domain Requirements and Business Rules

| ID | Rule | Source (policy, law, standard) |
|---|---|---|
| BR-01 | **Weekly Student Quota Limit**<br><br>Each student group or student ID is allowed to book a maximum cumulative total of 8 hours per calendar week. | Library Management Policy |
| BR-02 | **Daily & Single-Session Duration Limits**<br>A student group is restricted to a maximum reservation of 4 hours per session, and no more than 4 hours per day. | Library Management Policy |
| BR-03 | **Mandatory Booking Information Requirements**<br>Require every booking submission must provide the following data fields: (1) Booking Start & End Time, (2) Representative Student ID, (3) Total Member Count, and (4) Individual Student IDs of all group members. If any of these details are missing, the system shall reject or automatically cancel the booking request. | Library Management Policy |
| BR-04 | **Operating Hours & Slot Validity**<br><br>Rooms can only be reserved during official operational time slots. | Library Schedule Calendar |
| BR-05 | **Conflict Prevention (No Double-Booking)**<br><br>No study room can be reserved for overlapping time intervals. The system must enforce time-slot conflict checks (invoking *UC-04: Check Booking Conflict*) before confirming any reservation. | Software Specification Requirement |
| BR-06 | **Maintenance Overriding Rule**<br><br>When a Librarian sets a room to "Under Maintenance" (UC-09), all pending or approved reservations falling within the maintenance timeframe must be automatically cancelled and notifications dispatched. | Library Management Policy |

## 5.4 Acceptance Criteria

**Acceptance Criteria for F01: Basic Room Booking**

- AC-F01-01: Successful Room Booking Submission:
    - Given: an authenticated student has not exceeded their daily quota of 4 hours and weekly quota of 8 hours, and a target study room is available for the selected time slot.
    - When: the student selects the room, fills in all mandatory fields (Start/End Time, Representative Student ID, Member Count, and all Group Member Student IDs), and clicks Submit Booking.
    - Then: the system shall create a new booking record with status Pending, and dispatch a confirmation email to the student's registered email address.
- AC-F01-02: Rejection of Overlapping Time Slot Booking (BR-05):
    - Given: a study room already has an approved or pending reservation in working hours on a specific date.
    - When: another student attempts to submit a reservation for the same room from that same time on that date.
    - Then: the system shall invoke UC-04 (Check Booking Conflict) and reject the submission, and display an error message: The selected time slot conflicts with an existing reservation.
- AC-F01-03: Rejection Due to Incomplete Group Information (BR-04):
    - Given: an authenticated student is filling out the room reservation form.
    - When: the student attempts to submit the form without providing the list of individual group member Student IDs.
    - Then: the system shall block the form submission, and highlight the missing fields and show the error.
- AC-F01-04: Rejection Due to Exceeding Duration Quotas (BR-01 & BR-02):
    - Given: a student has already accumulated 6 hours of approved room bookings in the current calendar week.

- When: the student attempts to book a new session for 4 continuous hours result in 10 hour in this week.
- Then: the system shall validate the request against quota rules and prevent order placement, and display an alert (Weekly booking limit exceeded).

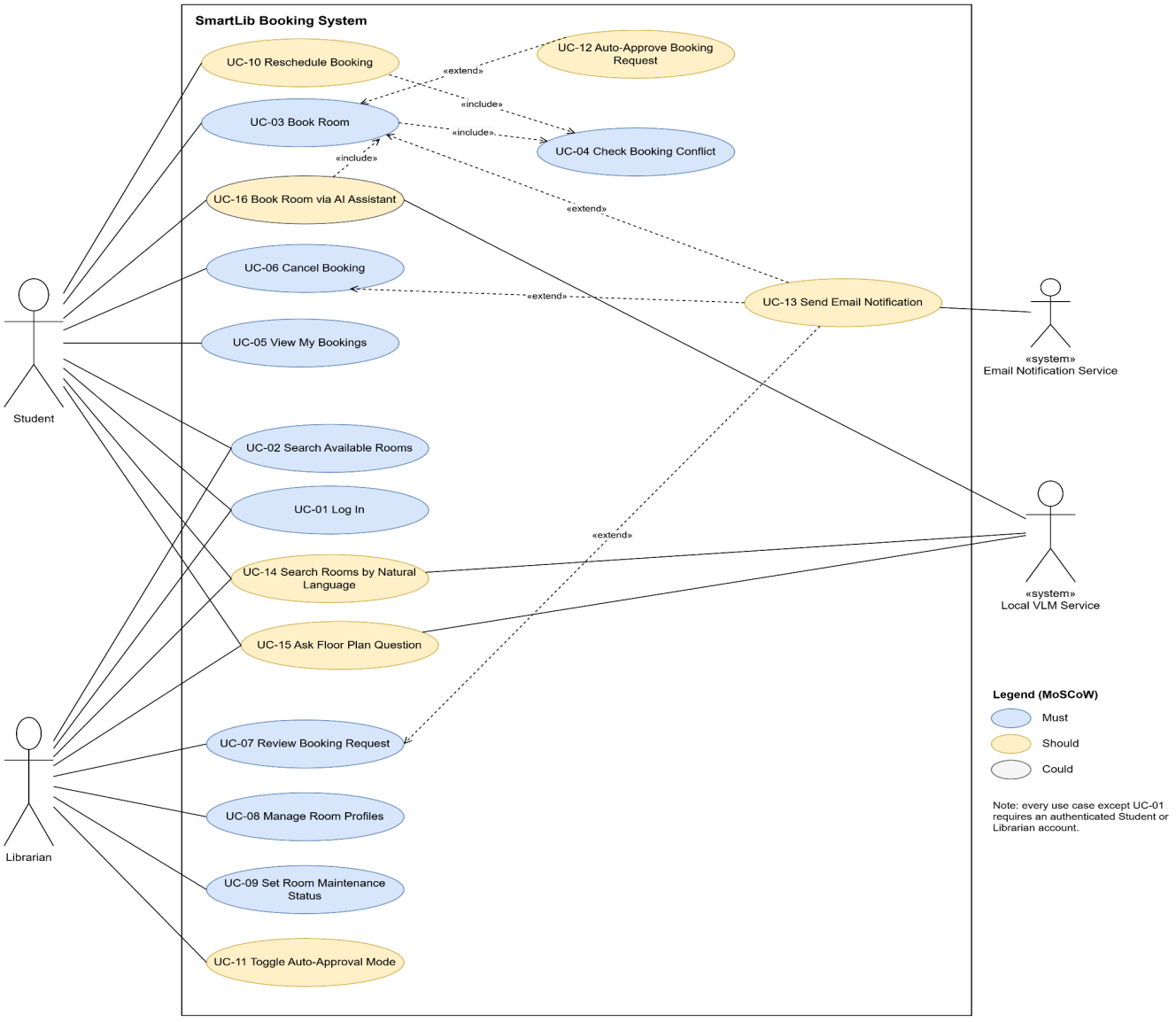
- AC-F01-05: Successful Booking Cancellation:
  - Given: a student has an upcoming reservation in Pending or Approved state scheduled to start in 2 hours.
  - When: the student views My Booking and clicks the Cancel Booking button.
  - Then: the system shall update the booking status to Cancelled, and immediately release the reserved time slot back to the public availability calendar.

**Acceptance Criteria for F02: Room Inventory Management**

- AC-F02-01: Adding a New Operational Room Profile:
  - Given: an authenticated Librarian is accessing the Room Inventory Management.
  - When: the Librarian inputs a unique Room ID, Capacity, Floor Location, and Available Amenities, and clicks Save Profile.
  - Then the system shall create the new room record in the database, and make the room immediately shown and searchable in the student booking interface.
- AC-F02-02: Setting Room Maintenance Status Overriding Existing Bookings (BR-06):
  - Given: a study room has an active approved student booking for tomorrow between 13:00 and 15:00.
  - When: a Librarian sets that room's status to Under Maintenance covering tomorrow's operational hours.
  - Then: the system shall flag the room as unavailable for search, and automatically set the student's booking status to Cancelled, and trigger an automated notification email informing the student of the maintenance closure.

# 6 Use Case Model

## 6.1 Use Case Diagram



## 6.2 Actors

| Actor | Description |
|---|---|
| Student | University student (primary actor) who searches for rooms, books them, and views, cancels or reschedules their own bookings. |
| Librarian | Library staff member (primary actor) who manages room inventory and maintenance status, reviews booking requests and controls the auto-approval switch. |
| Email Notification Service | External system (secondary actor) that sends emails when a booking is submitted, approved or canceled. |
| Local VLM Service | External system (secondary actor): the locally hosted Vision-Language Model that handles natural-language and floor-plan questions for the AI assistant. |
| Central Identity Provider | External university system responsible for managing student/staff account credentials. |

## 6.3 Use Case List

| ID | Use case | Primary actor | Priority | Related FR |
|---|---|---|---|---|
| UC-01 | Log In | Student, Librarian | Must | FR-01 |
| UC-02 | Search Available Rooms | Student, Librarian | Must | FR-02 |
| UC-03 | Book Room | Student | Must | FR-03 |
| UC-04 | Check Booking Conflict | Student, Librarian (included use case) | Must | FR-04 |
| UC-05 | View My Bookings | Student | Must | FR-05 |
| UC-06 | Cancel Booking | Student | Must | FR-06 |
| UC-07 | Review Booking Request | Librarian | Must | FR-07 |
| UC-08 | Manage Room Profiles (add, edit, disable) | Librarian | Must | FR-08 |
| UC-09 | Set Room Maintenance Status | Librarian | Must | FR-09 |
| UC-10 | Reschedule Booking | Student | Should | FR-10 |
| UC-11 | Toggle Auto-Approval Mode | Librarian | Should | FR-11 |
| UC-12 | Auto-Approve Booking Request | Student (initiates through UC-03) | Should | FR-12 |

| | | | | |
|---|---|---|---|---|
| UC-13 | Send Email Notification | Student, Librarian (indirect trigger) | Should | FR-13 |
| UC-14 | Search Rooms by Natural Language | Student, Librarian | Should | FR-14 |
| UC-15 | Ask Floor Plan Question | Student, Librarian | Should | FR-15 |
| UC-16 | Book Room via AI Assistant | Student | Should | FR-16 |

# 7 Use Case Specifications

## 7.1 Use Case UC-01

| Use case ID | UC-01 |
|---|---|
| Use case name | Log In |
| Brief description | Authenticates a Student or Librarian into the system using university credentials. |
| Primary actor | Student, Librarian |
| Secondary actors | Authentication Service Provider. |
| Priority | Must |
| Related requirements | FR-01 |
| Trigger | User navigates to the portal and clicks the "Log In" button. |
| Pre-conditions | User possesses an active university account and is not currently authenticated in the active session. |
| Post-conditions | Identity validated; An active session token is created, and role-based access control (Student or Librarian) is established. |
| Main flow | 1. User clicks the Log In button on the navigation bar.<br>2. System displays the Central Authentication Login page.<br>3. User inputs their ID / Email and Password, then submits the form.<br>4. System validates the credentials against the user database identity provider.<br>5. System retrieves user role permissions (Student or Librarian).<br>6. System initializes the user session and redirects the user to their respective dashboard. |
| Alternative flows | None |
| Exception flows | **E1 (at step 4): Invalid Credentials**<br>1. System detects wrong ID or password.<br>2. System displays an error message: "Invalid username or password."<br>3. Flow returns to step 3. |
| Business rules | None |
| Non-functional constraints | Authentication latency shall be under 500ms. |

## 7.2 Use Case UC-02

| Use case ID | UC-02 |
|---|---|
| Use case name | Search Available Rooms |
| Brief description | Allows users to view, and filter library study rooms based on date, time slot, capacity,... |
| Primary actor | Student, Librarian |
| Secondary actors | None |
| Priority | Must |
| Related requirements | FR-02 |
| Trigger | User accesses the "Find a Room" page. |
| Pre-conditions | User is authenticated into the system. |
| Post-conditions | A filtered list of available rooms is displayed. |
| Main flow | 1. User selects the desired booking date, optional filters (room capacity,...).<br>2. User clicks the "Search" button.<br>3. System queries existing room schedules.<br>4. System displays a grid/list of available rooms matching the criteria along with room details.<br>5. User selects a room card to view detailed availability slots |
| Alternative flows | A1 (at step 1): Search Without Filters<br>1. User submits the search without specifying extra criteria.<br>2. System returns all operational rooms available for the default time slot (current hour). |
| Exception flows | E1 (at step 3): No Available Rooms<br>1. The system finds zero matching rooms available for the requested time frame.<br>2. System renders a notification: "No rooms available for the selected time slot." and suggests alternative nearby time slots. |
| Business rules | The booking room can only be searched within official library opening hours. (BR-04) |
| Non-functional constraints | Search queries must return results in ≤ 1s under normal server load. |

## 7.3 Use Case UC-03

| Use case ID | UC-03 |
|---|---|
| Use case name | Book Room |
| Brief description | Allows a student to create a room reservation for a specific time slot . |

| | |
|---|---|
| Primary actor | Student |
| Secondary actors | Email Notification Service |
| Priority | Must |
| Related requirements | FR-03 |
| Trigger | Student selects an available time slot and clicks "Book Now". |
| Pre-conditions | Student is authenticated and has not exceeded daily quota limits (8 hour/ week, 4 hour/ day) |
| Post-conditions | A new booking record is created in "Pending" or "Approved" state, reserving the room slot. |
| Main flow | 1. Student selects a room and an available time slot from the search results.<br>2. Student enters booking purpose and attendee count, then clicks Confirm Booking.<br>3. System invokes UC-04: Check Booking Conflict to verify slot availability.<br>4. System validates student quota constraints.<br>5. System persists the booking record with status "Pending Review" (or "Approved" if auto-approval is ON).<br>6. System triggers Email Notification Service to send a confirmation mail.<br>7. System displays booking confirmation to the student. |
| Alternative flows | A1(at step 5): Auto-Approve Booking<br><br>1. System detects that Auto-Approval Mode (UC-11) is active.<br>2. System sets booking status directly to "Approved". |
| Exception flows | E1 (at step 3): Conflict Detected<br><br>1. UC-04 reports a time slot overlap.<br>2. System cancels the process and alerts student: "This slot was just booked by another user. Please choose another time."<br><br>E2 (at step 4): Quota Exceeded<br><br>1. System detects student has reached the daily limit of 4 hours a day or 8 hours a week.<br>2. System displays error: "Daily booking limit reached" |
| Business rules | BR-01, BR-02, BR-03, BR-04, BR-05 |
| Non-functional constraints | Booking transactions must enforce database atomicity. |

## 7.4 Use Case UC-04

| Use case ID | UC-04 |
|---|---|

| | |
|---|---|
| Use case name | Check Booking Conflict |
| Brief description | Validates whether a proposed room booking or reschedule overlaps with any existing approved/pending booking or room is in maintenance. |
| Primary actor | Student, Librarian |
| Secondary actors | Database Management System |
| Priority | Must |
| Related requirements | FR-04 |
| Trigger | Included automatically by UC-03 (Book Room) or UC-10 (Reschedule Booking) |
| Pre-conditions | Room ID, proposed Start Time, and End Time. |
| Post-conditions | Return the conflict condition to the calling parent use case |
| Main flow | 1. System receives Room ID, Start Time, and End Time from the parent flow.<br>2. System locks room record temporarily to prevent race conditions.<br>3. System queries the database for existing active bookings for the same Room ID where ProposedStart < ExistingEnd AND ProposedEnd > ExistingStart.<br>4. System checks if the room has an active "Maintenance" flag set for that period (UC-09).<br>5. System verifies no time overlap exists and returns Conflict: False to the parent flow. |
| Alternative flows | None |
| Exception flows | E1 (at step 5): Overlap Found<br>1. System identifies an existing booking or maintenance block in the time slot.<br>2. System returns Conflict: True and details of the conflicting time range to the parent use case. |
| Business rules | BR-05 |
| Non-functional constraints | Execution time must be ≤ 100ms. |

## 7.5 Use Case UC-05

| Use case ID | UC-05 |
|---|---|
| Use case name | View My Bookings |
| Brief description | Displays a historical and upcoming list of room reservations made by the logged-in student |
| Primary actor | Student |
| Secondary actors | None |
| Priority | Must |
| Related requirements | FR-05 |

| | |
|---|---|
| Trigger | Student clicks "My Bookings" in the account navigation menu. |
| Pre-conditions | Student is authenticated. |
| Post-conditions | Current, past, and cancelled reservations are displayed. |
| Main flow | 1. Student selects "My Bookings" menu.<br>2. System retrieves all booking records associated with the logged-in Student ID.<br>3. System show bookings into "Upcoming", "Completed", and "Cancelled" tabs.<br>4. System renders the list showing Room Name, Date, Time Slot, Status, and Action buttons (Cancel/Reschedule).<br>5. Student clicks a booking card to view complete booking details. |
| Alternative flows | None |
| Exception flows | E1 (at step 2): No Bookings Found<br>1. System finds 0 records for the student.<br>2. System renders an empty state page with a link: "You have no bookings. Click here to search for a room." |
| Business rules | BR-03 |
| Non-functional constraints | Page load time ≤ 1s |

## 7.6 Use Case UC-06

| Use case ID | UC-06 |
|---|---|
| Use case name | Cancel Booking |
| Brief description | Allows a student to cancel an upcoming pending or approved room reservation. |
| Primary actor | Student |
| Secondary actors | Email Notification Service |
| Priority | Must |
| Related requirements | FR-06 |
| Trigger | Student clicks Cancel Booking next to the reservation in My Bookings. |
| Pre-conditions | Student is authenticated; booking status is Pending or Approved. |
| Post-conditions | Booking status is updated to "Cancelled", freeing up the room time slot in the calendar. |
| Main flow | 1. Student navigates to "My Bookings" (UC-05) and clicks "Cancel Booking" on an upcoming reservation.<br>2. System prompts a confirmation form asking for a cancellation reason.<br>3. Student confirms the cancellation.<br>4. System updates booking record status to Cancelled in the database.<br>5. System releases the reserved time slot back. |

| | |
|---|---|
| | 6. System invokes Email Notification Service to send a cancellation notice to the student.<br>7. System updates UI display and shows a success notification. |
| Alternative flows | None |
| Exception flows | None |
| Business rules | None |
| Non-functional constraints | Slot release in the search index must occur synchronously within 1 second. |

## 7.7 Use Case UC-07

| Use case ID | UC-07 |
|---|---|
| Use case name | Review Booking Request |
| Brief description | Allows a librarian to view pending booking requests and manually approve or reject them. |
| Primary actor | Librarian. |
| Secondary actors | Email Notification Service. |
| Priority | Must. |
| Related requirements | FR-07 |
| Trigger | Librarian opens the Pending Requests tab in the administrative dashboard. |
| Pre-conditions | Librarian is logged in; Auto-Approval Mode is turned OFF. |
| Post-conditions | Booking request state changes to Approved or Rejected with notification dispatched. |
| Main flow | 1. Librarian selects "Pending Booking Requests" from the dashboard menu.<br>2. System lists all pending requests sorted by submission timestamp.<br>3. Librarian clicks on a request to view student details, requested room, and purpose.<br>4. Librarian clicks "Approve".<br>5. System updates booking status to "Approved".<br>6. System triggers Email Notification Service to send an approval confirmation email with check-in details to the student.<br>7. System removes the item from the pending list. |
| Alternative flows | A1 (at step 4): Reject Request<br><br>Librarian clicks "Reject" and enters a rejection reason.<br><br>System updates booking status to Rejected.<br><br>System triggers Email Notification Service to notify the student with the reason included. |

| | |
|---|---|
| Exception flows | E1 (at step 4): Request Already Processed<br>1. System detects another librarian approved/rejected the request.<br>2. System alerts: This request has already been processed, and refreshes list. |
| Business rules | None |
| Non-functional constraints | Administrative queue updates must be reflected in real time. |

## 7.8 Use Case UC-08

| Use case ID | UC-08 |
|---|---|
| Use case name | Manage Room Profiles |
| Brief description | Allows a librarian to create, edit, disable, or configure room data(capacity, status, location). |
| Primary actor | Librarian |
| Secondary actors | File Storage (for floor plans/photos). |
| Priority | Must |
| Related requirements | FR-08 |
| Trigger | Librarian accesses the Room Inventory Management section. |
| Pre-conditions | Librarian is authenticated with administrative privileges. |
| Post-conditions | Room profile record is added, updated, or soft-deleted in the database. |
| Main flow | 1. Librarian enters Room Management dashboard.<br>2. System displays a list of all library study rooms.<br>3. Librarian clicks Add New Room (or Edit on an existing room).<br>4. Librarian inputs Room description (Number, Floor, Capacity, Available Equipment (TV, PC, Board), and uploads photo).<br>5. Librarian clicks Save Profile.<br>6. System validates room inputs.<br>7. System saves room data to the database.<br>8. System displays confirmation message and updates the room list. |
| Alternative flows | A1 (at step 3): Disable Room Profile<br>1. Librarian selects "Disable Room".<br>2. System marks room status as "Inactive" (hides it from student search results without deleting history). |
| Exception flows | E1 (at step 6): Duplicate Room Number<br>1. System detects the room number already exists on that floor.<br>2. System flags the field error: "Room number already exists." |

| | |
|---|---|
| Business rules | Active rooms with future bookings cannot be hard-deleted from the database. |
| Non-functional constraints | Room profile updates take immediate effect across search endpoints. |

## 7.9 Use Case UC-09

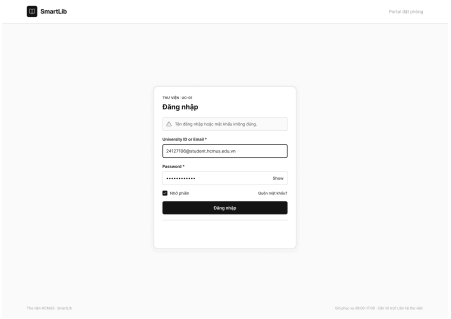
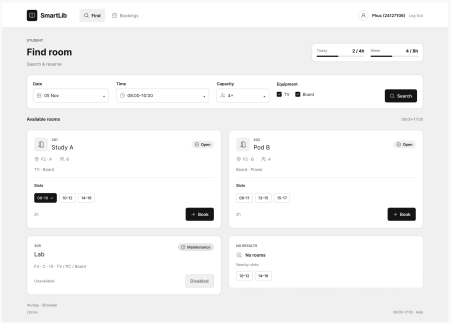
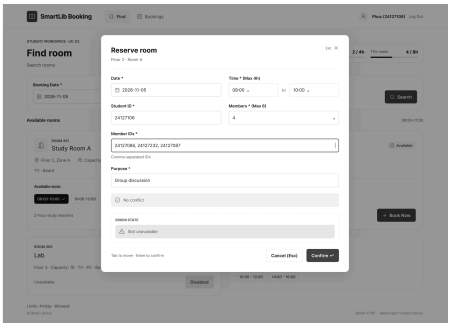
| Use case ID | UC-09 |
|---|---|
| Use case name | Set Room Maintenance Status |
| Brief description | Allows a librarian to mark a room as Under Maintenance (or something else) for a specified date range. |
| Primary actor | Librarian. |
| Secondary actors | Email Notification Service |
| Priority | Must |
| Related requirements | FR-09 |
| Trigger | Librarian selects Schedule Maintenance on a room profile. |
| Pre-conditions | Librarian is logged in. |
| Post-conditions | Room is flagged as unavailable for bookings during the specified maintenance time. |
| Main flow | 1. Librarian selects a room from the inventory list and clicks "Set Maintenance Status".<br>2. Librarian enters Maintenance Reason, Start Date/Time, and End Date/Time.<br>3. Librarian clicks Apply Maintenance.<br>4. System checks if any existing bookings fall within the maintenance window.<br>5. System flags the room as Under Maintenance for the period.<br>6.<br>7. System updates room status in the search database.<br>8. System displays confirmation alert. |
| Alternative flows | A1 (at step 4): Handle Conflicting Bookings<br>1. System identifies 2 pending/approved student bookings during the maintenance time.<br>2. System updates those booking statuses to Cancelled by Library.<br>3. System triggers Email Notification Service to email affected students with the maintenance reason and prompt them to rebook. |
| Exception flows | E1 (at step 3): Invalid Date Range<br>1. End time is set before Start time.<br>2. System shows error: "End date must be after Start date." |
| Business rules | BR-06 |

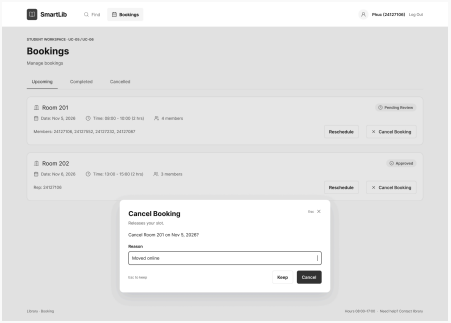
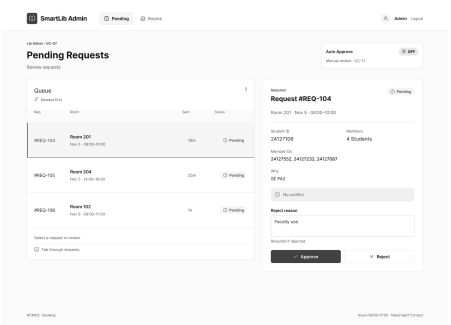
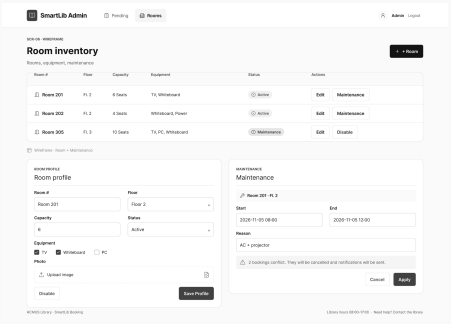
| | |
|---|---|
| Non-functional constraints | Cancellation emails to affected users must be queued within 1 hour. |

# 8 Prototype / Mockups

**Figma Design Workspace:**

https://www.figma.com/design/4zQzpj1MTeNU4ysWRSiFIE/Wireframes?node-id=0-1&t=VQTEAjdVnpfG9Rgu-1

| Screen ID | Screen Name | Related use case(s) | Mockup / link |
|---|---|---|---|
| **SCR-01** | Central Login Page | **UC-01 (Log In)** |  |
| **SCR-02** | Find a Room (Search Dashboard) | **UC-02 (Search Available Rooms)** |  |
| **SCR-03** | Room Booking & Conflict Check Modal | **UC-03 (Book Room), UC-04 (Check Booking Conflict)** |  |

| | | | |
|---|---|---|---|
| **SCR-04** | My Bookings & Cancellation Modal | **UC-05 (View My Bookings), UC-06 (Cancel Booking)** |  |
| **SCR-05** | Pending Booking Requests Queue | **UC-07 (Review Booking Request)** |  |
| **SCR-06** | Room Inventory & Maintenance Management | **UC-08 (Manage Room Profiles), UC-09 (Set Room Maintenance Status)** |  |

# 9 Requirements Traceability Matrix

| Req. ID | Use case | Screen | Design element (PA3) | Test cases (PA4) |
|---|---|---|---|---|
| FR-01 | UC-01 | SCR-01: Central Login Page | TBD | TBD |
| FR-02 | UC-02 | SCR-02: Find a Room (Search Dashboard) | TBD | TBD |
| FR-03 | UC-03 | SCR-03: Room Booking & Conflict Check Modal | TBD | TBD |
| FR-04 | UC-04 | SCR-03: Room Booking & Conflict Check Modal | TBD | TBD |
| FR-05 | UC-05 | SCR-04: My Bookings & Cancellation Modal | TBD | TBD |
| FR-06 | UC-06 | SCR-04: My Bookings & Cancellation Modal | TBD | TBD |
| FR-07 | UC-07 | SCR-05: Pending Booking Requests Queue | TBD | TBD |
| FR-08 | UC-08 | SCR-06: Room Inventory & Maintenance Management | TBD | TBD |
| FR-09 | UC-09 | SCR-06: Room Inventory & Maintenance Management | TBD | TBD |
| FR-10 | UC-10 | SCR-04: My Bookings & Cancellation Modal | TBD | TBD |
| FR-11 | UC-11 | TBD | TBD | TBD |
| FR-12 | UC-12 | SCR-03: Room Booking & Conflict Check Modal | TBD | TBD |
| FR-13 | UC-13 | TBD | TBD | TBD |
| FR-14 | UC-14 | TBD | TBD | TBD |
| FR-15 | UC-15 | TBD | TBD | TBD |

| | | | | |
|---|---|---|---|---|
| FR-16 | UC-16 | TBD | TBD | TBD |

**Notes:**

- TBD: To Be Determined. Design elements and test cases will be added in PA3 and PA4, respectively.

# 10 Requirements Review

A requirements review meeting was held prior to the submission of PA2.

- Review Date: November 3, 2026.
- Participants: All team members
- Outcome: All identified issues were discussed, resolved and add to baseline version of the SRS document.

| # | Issue found | Requirement(s) | Resolution |
|---|---|---|---|
| 1 | Unclear Handling of Pre-existing Bookings during Maintenance: The initial spec did not detail what happens to approved student bookings when a room is set to Under Maintenance by staff | FR-09, BR-06, UC-09 | Added BR-06 and updated UC-09 Main/Alternative flows: setting a room to maintenance automatically cancels conflicting student bookings and queues notification emails. |
| 2 | Missing Mandatory Group Member ID: Booking specifications initially only required the representative student's ID. | FR-03, BR-03, AC-F01-03 | Updated BR-03, UC-03, and AC-F01-03 to make Individual Student IDs of all group members a mandatory field for form submission. |

# 11 AI Usage Log

| # | Member | Tool | Task / purpose | How the output was verified and changed |
|---|---|---|---|---|
| 1 | Nguyễn Võ Nhật Thuận | Gemini | Draft Acceptance Criteria in section 5.4 ,using the Given/When/Then method for Must-have features. | Verified scenarios against project scope; ensured constraints alignment with business rules BR-01 to BR-06 (quotas, non-overlapping slots, mandatory fields, and auto-cancellation). |
| 2 | Nguyễn Võ Nhật Thuận | Gemini | Draft Use Case Specifications in section 7 for all 9 Must-have Use Cases from UC-01 to UC-09 following the PA2 specification template. | Cross-checked Use Case IDs and names, and FR against the SmartLib use case diagram; verified pre-conditions, main flows, alternative flows, exception flows, and business rule mappings matched project requirements. |
| 3 | Dương Nguyên Nam | Claude | Drafted Section 6 (Use Case Model): the use case diagram as a draw.io file, the actor table and the use case list with MoSCoW priorities. | Checked each use case against the PA1 features F01 to F05 and the problem statement. Redraw and re-laid out the diagram in draw.io. Removed the abstract "Library User" actor and its generalization arrows, and connected Student and Librarian directly to the shared use cases. Changed UC-16 priority from Could to Should. Renumbered the related FR IDs to match Section 5.1. |
| 4 | Dương Nguyên Nam | Claude | Drafted Section 4, keeping the five stakeholders from PA1 and linking their interests to the use cases and business rules in this SRS. | Checked that the stakeholder names and order match PA1 Section 2.3. Verified every UC and BR reference against Sections 5.3, 6 and 7. |
| 5 | Lai Phạm Như Phúc | Gemini | Plan screen consolidation, ASCII wireframe layouts, and accessibility checklist (contrast, labels, keyboard focus) for Section 8 Prototype / Mockups covering UC-01 to UC-09. | Verified screen flows against all 9 Must-have use case specifications and Business Rules (BR-01 to BR-06). Manually designed the UI mockups and interactive prototype connections in Figma, verified WCAG contrast ratios, and replaced text wireframes with actual Figma screenshots. |

| | | | | |
|---|---|---|---|---|
| 6 | Lê Phụng Sơn | Gemini | Drafted the SRS Introduction, including Purpose and Scope and Glossary, based on the SmartLib Booking project requirements. | Cross-checked the purpose, scope, and terminology against PA1 to ensure they reflected the documented system features and used consistent terms. Removed unsupported details. |
| 7 | Lê Phụng Sơn | Gemini | Drafted Functional Requirements (FRs) and Non-Functional Requirements (NFRs) for the SRS. | Verified the FRs against the vision and proposed solution sections in PA1. Checked NFRs against the documented performance, consistency, reliability, and notification constraints. |
| 8 | Trần Hoàng Khánh My | ChatGPT | Draft Problem Statement in Section 3, elaborating the preliminary problem statement from PA1, including business context, current problems, goals, operating environment, and implementation constraints. | Cross-checked the content against PA0 and PA1; refined the problem description and ensured consistency with SmartLib Booking's scope, technology stack, and business rules defined in PA2. |